



AWS Solution Architect

(UNIT-4: Database, Management Tools, Management & Governance)

Dr. Shagun Sharma

Assistant Professor

School of Computing Science and Engineering

shagunsharma@vitbhopal.ac.in

RDS Multi-AZ Deployments



Amazon RDS (Relational Database Service)

- Amazon Relational Database Service (Amazon RDS) is a managed service provided by Amazon Web Services (AWS) that simplifies the setup, operation, and scaling of relational databases in the cloud. It automates many of the time-consuming database administration tasks, allowing users to focus on their applications and business needs.
- RDS Multi-AZ deployment is an Amazon service that enhances database availability by creating a synchronous standby replica of your primary database in a different Availability Zone (AZ). This provides automatic failover in the event of a primary database or AZ failure, protecting your data with zero data loss and no manual intervention. While the primary instance handles all read and write traffic, the standby replica is kept up-to-date via synchronous replication and is only activated during a failover to become the new primary.

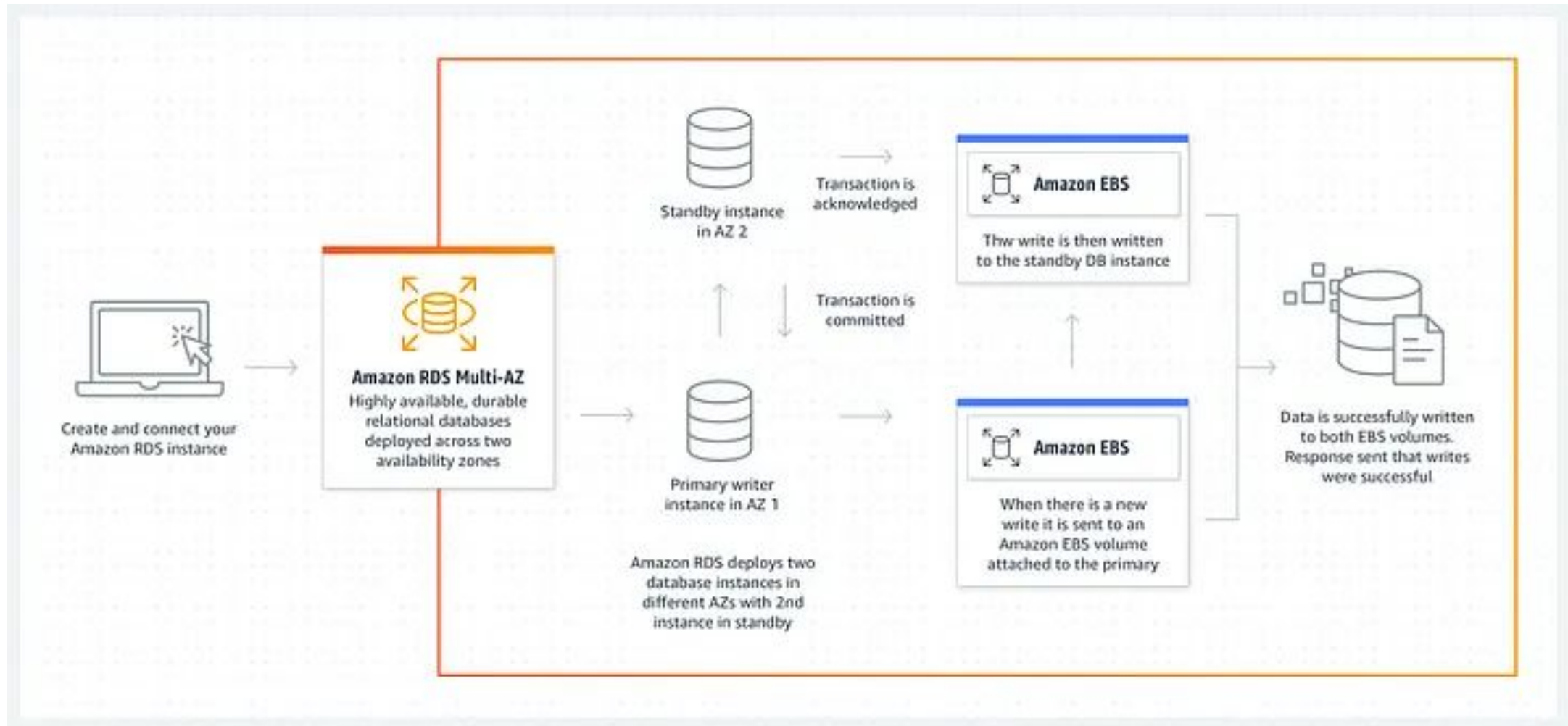
Multi-AZ deployments

RDS Multi-AZ Deployment is like having a backup copy of your database automatically running in a different, separate location within the same region.

Imagine your main database is in one building (an Availability Zone). With Multi-AZ, Amazon RDS automatically creates and maintains an exact, synchronized copy of that database in a completely different building (another Availability Zone).

RDS Multi-AZ Deployments

The following diagram shows a Multi-AZ deployment with one standby DB instance, and how it works.



Key Features of RDS Multi-AZ Deployments



- **Managed Service:** AWS handles tasks such as provisioning, patching, backups, and operating system updates.
- **Database Engine Support:** Supports popular relational database engines including PostgreSQL, MySQL, MariaDB, Oracle, SQL Server, and Amazon Aurora (a cloud-native, MySQL and PostgreSQL-compatible database).
- **Scalability:** Allows for easy scaling of compute resources and storage capacity to meet changing demands.
- **High Availability and Durability:** Offers features like Multi-AZ deployments for automatic failover and read replicas for improved data durability and read-heavy workloads.
- **Security:** Provides various security features, including network isolation within Amazon Virtual Private Cloud (VPC), encryption at rest and in transit, and integration with AWS Identity and Access Management (IAM).
- **Cost-Effective:** Offers a pay-as-you-go model with no upfront investments, and various pricing options like On-Demand and Reserved Instances.

RDS Multi-AZ Deployments



Key Features of RDS Multi-AZ

- **Synchronous Replication:**
 - Changes made to the primary DB are immediately replicated to the standby DB.
- **Automatic Failover:**
 - If the primary instance fails (hardware failure, AZ outage, maintenance, etc.), RDS automatically fails over to the standby without manual intervention.
- **No Manual Synchronization Needed:**
 - AWS handles all the replication and synchronization transparently.
- **Same Endpoint:**
 - Applications continue using the same DB endpoint even after failover, minimizing downtime.
- **Backup from Standby:**
 - Backups are taken from the standby instance to reduce I/O on the primary.

RDS Read Replicas



- Customers can also make RDS more highly available by using read replicas.
- Amazon RDS read replicas provide enhanced performance and durability for Amazon RDS DB instances. For read-heavy database workloads, read replicas make it easier to elastically scale out beyond the capacity constraints of a single DB instance.
- You can create one or more replicas of a given source DB instance and serve high-volume application read traffic from multiple copies of your data, which increases aggregate read throughput. Read replicas can also be promoted to become standalone DB instances, when needed.
- Read replicas are available in Amazon RDS for MySQL, MariaDB, PostgreSQL, Oracle, Microsoft SQL Server, and Amazon Aurora.
- For the MySQL, MariaDB, PostgreSQL, Oracle, and SQL Server database engines, Amazon RDS creates a second DB instance by using a snapshot of the source DB instance. Amazon RDS then uses the engine's native asynchronous replication to update the read replica when there's a change to the source DB instance.
- The read replica operates as a DB instance that allows only read-only connections. Applications can connect to a read replica like they would connect to any DB instance. Amazon RDS replicates all databases in the source DB instance.

RDS Read Replicas



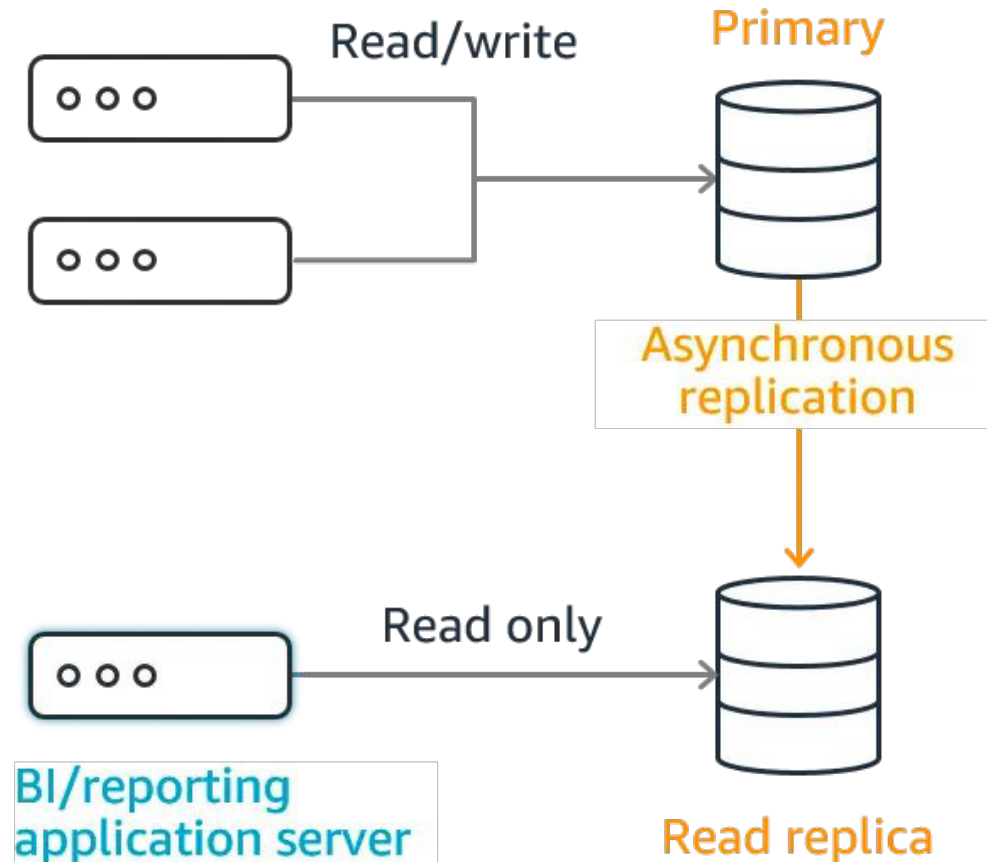
- RDS Read Replicas are read-only copies of a primary database that handle read-heavy workloads, like analytics, to prevent the primary database from getting overloaded.
- Think of it like having a main book (the primary database) and several extra copies that people can read from, while the main book is only used for new writing (writes).
- This helps improve overall performance and scalability because the main database can focus on writes without being slowed down by too many readers.

RDS Read Replicas



Here's an example of when to use a read replica. Say that you're running reports on your database, which is causing performance issues with CPU-intensive reads. You can use a read replica and direct all the reporting queries to that replica instead of to the primary instance. Offloading some of the intense queries to the replica should result in enhanced performance on the primary instance.

Application servers Database server



RDS Read Replicas



Advantages of Read Replicas

- Gives an ability to scale read operation in relational databases in order to support read-intensive workloads.
- Read replicas can be promoted to their own standalone database instance at any time.
- Reduces the data access time by deploying the read replicas in multiple regions.
- You can reduce the load of the source DB by routing the read queries to read replicas.

Disadvantages of Read Replicas

- Read Replicas supports only select clause queries.
- The data which has been created (or) modified at the source DB takes some time to get updated at the read replicas since they use asynchronous mode of replication.
- Charges are applied if the read replicas are present in different regions.

RDS Read Replicas



How Is Read Replicas Differ From Multi-AZ Deployment

- RDS Multi AZ deployments are mainly used for backups and disaster recovery strategies. Multi AZ deployments contain a primary database and a standby database, when the primary database goes down (or) becomes unavailable the standby database becomes active.
- While read replicas are used to scale reads in a database, multi-AZ deployments are used to increase availability of data. Multi-AZ deployments uses synchronous mode of replication.
- **SYNCHRONOUS REPLICATION** : In synchronous replication data will be written to two or more systems simultaneously.

How does RDS Read Replicas work?



- **Primary database:** This is the main source where all data is written and updated.
- **Read replicas:** These are duplicate copies of the primary database. All new data from the primary is copied to the read replicas in an asynchronous process.
- **Reading and writing:** Applications can be configured to send all read queries (like "show me this report") to the read replicas, while all write operations (like "add this new record") are sent only to the primary database.
- **Scalability:** By distributing read traffic across multiple replicas, you can scale your database's read capacity far beyond what a single instance could handle.
- **Disaster recovery:** A read replica can be promoted to become a new, standalone primary database if the original instance fails.

RDS Read Replicas



ATTRIBUTES	RDS READ REPLICAS	RDS MULTI-AZ DEPLOYMENT
PURPOSE	Used to scale reads in a database by creating multiple read only databases.	Used to increase availability of a database by creating an exact replica of the entire database and deploying it in another AZ.
MODE OF REPLICATION	Used asynchronous mode of replication.	Used synchronous mode of replication.
SUPPORTED OPERATIONS	Other than source databases, all the read replicas only support select clause queries.	Both primary and standby databases support all types of queries.
NO. OF DUPLICATE DB INSTANCES	Supports up to 15 read replicas for a single DB instance.	Supports only 2 databases: a primary database and a secondary database.
OTHERS	Read replicas can be promoted to its own databases.	The standby database becomes active only when the primary database is not available.

Amazon Aurora



- Amazon Aurora is a high-performance, cloud-based database service from Amazon Web Services (AWS) that is compatible with MySQL and PostgreSQL.
- In simple terms, it's a super-fast, highly reliable database built for the cloud, combining the best of expensive commercial databases with the ease of open-source ones, but it's fully managed by Amazon.

What it is?

- **A cloud-native database:** It's designed specifically for the cloud, making it efficient and scalable.
- **MySQL and PostgreSQL compatible:** You can use your existing applications and tools that work with MySQL or PostgreSQL, as Aurora is built to be compatible with them.
- **High performance:** It offers significantly higher throughput (speed) than standard MySQL and PostgreSQL, often up to 5 times faster for MySQL and 3 times for PostgreSQL.

Amazon Aurora



- The **Amazon Aurora** is a relational database service offered from amazon cloud. This is one of the widely used services for the data storage, for low latency and value-based data storage and processing. Amazon Aurora is a MySQL and PostgreSQL-compatible relational database fabricated for the cloud, that consolidates the performance and accessibility of traditional databases with the simplicity and reliability of commercial databases at 1/10th the cost. It works with a clustered approach with data replication in the AWS accessibility zone for efficient data availability.
- The Amazon Aurora is packed with high-performance subsystems. It is MySQL and PostgreSQL engines which take advantage of fast distributed storage. Aurora provides speed and throughput up to 5 times of MySQL and 3 times of PostgreSQL with the existing system. It bolsters, high storage capacity, which can scale up to 64 Terabytes of database size for enterprise implementation. Amazon Aurora is completely managed by Amazon Relational Database Service (RDS), which automates tedious administration undertakings like hardware provisioning, database arrangement, fixing, and reinforcements.

Amazon Aurora



- Amazon Aurora is built on top of an innovative **distributed architecture** that separates the storage and compute layers of the database engine. The storage layer is distributed across multiple replicas, while the compute layer runs on instances that are separate from the storage layer. This architecture allows for automatic scaling of storage and compute resources independently and also provides better fault tolerance and availability.

Who uses Amazon Aurora?



- **High-traffic websites and apps:** For applications that need to handle a lot of users or transactions at once, like e-commerce sites.
- **Enterprise applications:** Businesses use it for critical systems like Customer Relationship Management (CRM) or Enterprise Resource Planning (ERP) where reliability and security are crucial.
- **Data analysis:** It's also used for data warehousing and other applications where you need to quickly analyze large amounts of data.

Amazon Aurora



Features of Amazon Aurora

- **Availability and Durability:** AWS Aurora has a feature of fault-tolerant and self-mending storage built for the cloud. It offers an incredible accessibility of 99.99%. The storage in the cloud replicates the 6 copies of the information across 3 Availability Zones. The AWS Aurora backs up the data continuously due to the safety purpose and precaution from storage failure.
- **Performance and Scalability:** AWS Aurora provides 5 times the throughput of ordinary MySQL. This performance is comparable with enterprise databases, at 1/10th the cost. The user can scale database preparation up and down for smaller to larger instance varieties as per the user needs. To scale scan capacity and performance, the user can add up to fifteen low latency scan replicas across 3 convenience Zones. Amazon Aurora consequently develops storage as required, up to 64TB per database instance.
- **Fully Managed:** The Amazon Aurora is managed by Amazon Relational Database Service (RDS). The user no longer needs to stress over database management tasks, for example, hardware provisioning, software fixing, setup, configuration, or backups. Aurora consequently and consistently screens and backs up the database to Amazon S3, empowering granular point-in-time recuperation.

Amazon Aurora



Features of Amazon Aurora

- **Security:** Amazon Aurora provides numerous degrees of security to the database to improve it among others. On an encoded Amazon Aurora occurrence, data within the underlying storage is encrypted. The administration is through AWS Key Management Service and encryption of information in transit using SSL. In addition, there are the automatic reinforcements, snapshots, and replica within the same cluster.
- **Migration Support:** MySQL and PostgreSQL compatibility make Amazon Aurora a convincing target for database relocations to the cloud. If the users want to migrate from MySQL or PostgreSQL, can see migration documentation for a list of tools and options. To move from commercial database engines, the user can use the AWS Database Migration Service for a safe migration with minimal downtime.
- **Compatibility with MySql and PostgreSQL:** The Amazon Aurora database engine is perfectly compatible with existing MySQL and PostgreSQL open supply databases, and adds compatibility for new releases frequently. This means that the user can relocate MySQL or PostgreSQL databases to Aurora using standard MySQL or PostgreSQL import/export tools or previews. It also means the user using code, applications, drivers, and tools with existing databases can also use it with Amazon Aurora with little or no modification.
- **Cost:** Amazon Aurora is designed to be cost-effective. You only pay for what you use, and you can scale your database up or down as needed. Additionally, Aurora provides cost-saving features such as automated storage optimization and the ability to pause or stop your database when it's not in use.

Amazon Aurora



How does Amazon Aurora Work

- **A powerful cloud database:** Think of it as a supercharged, cloud-native database. It's not a traditional database you install on your own server; instead, you rent access to it over the internet through Amazon's cloud service.
- **Like MySQL or PostgreSQL, but faster:** It has been engineered to work seamlessly with applications built for MySQL and PostgreSQL, so you can often migrate existing databases without major changes. However, its special architecture allows it to handle up to 5 times the transactions of standard MySQL and 3 times that of PostgreSQL.
- **Handles the hard stuff for you:** Aurora is a "fully managed" service. This means Amazon handles many time-consuming tasks like setting up the database, patching it, backing it up, and making sure it stays available, so your team doesn't have to.
- **Always on and scalable:** It's built to be highly available, with data automatically copied across multiple data centers for reliability. It can also automatically scale its storage and power up or down to handle your needs without you having to do anything, ensuring performance during traffic spikes.

Amazon Aurora



How Amazon Aurora works with Amazon RDS

- In the Amazon RDS Shared Responsibility Model AWS and the customer share duties to ensure the security availability and performance of database services. AWS manages the infrastructure while customers take control of their data and database management.
- **Managed via Amazon RDS:** Aurora utilizes the Amazon RDS platform for administrative tasks such as provisioning patching backups and recovery. This management is performed through the AWS Management Console AWS CLI and API and allows developers and system administrators to focus on building and running their applications rather than dealing with underlying infrastructure management.
- **Operations Based on Clusters:** Unlike standard RDS instances Aurora operates on entire clusters of database servers that are automatically replicated. This architecture ensures high availability easy scaling and efficient resource management
- **High Availability:** Aurora replicates data across multiple Availability Zones for fault tolerance and automatic failover is handled by RDS in case of any failure.
- **Automated Scalability with Aurora:** Automated Scalability Aurora takes advantage of RDS automatic scaling capabilities which enable it to adjust storage and compute resources dynamically based on real-time workload demands.
- **Seamless Data Migration Migrating:** Seamless Data Migration Migrating from Amazon RDS for MySQL or PostgreSQL to Aurora is simple. You can use Amazon RDS snapshots or set up one-way replication to transfer your data smoothly. This allows you to benefit from Aurora's improved performance, scalability, and reliability without interrupting existing workflows.
- **DB Engine Selection:** When setting up a new database in Amazon RDS, users can opt for Aurora MySQL or Aurora PostgreSQL as the engine of choice. This offers the same familiarity as using traditional MySQL or PostgreSQL engines but with Aurora's performance boosts and reliability features.

Amazon Aurora

Amazon Aurora Pricing



- Amazon Aurora uses a pay-as-you-go pricing model which means you only pay for the resources you actually use. Here is a quick overview of the main factors that influence Aurora's pricing.
- **Instance Pricing:** Aurora charges based on the instance type and size you choose, with different prices for MySQL and PostgreSQL compatible instances. Larger instances cost more, but they also provide higher performance.
- **Storage Costs:** Aurora scales storage automatically according to your usage and you are charged per gigabyte of storage used. The benefit here is you only pay for the storage you need.
- **Backup Storage:** Aurora includes automated backups at no additional charge for up to the same amount of storage as your database. Additional backup storage is charged per GB.
- **I/O Requests:** You are billed for the input/output operations (I/O requests) performed by your database. Aurora offers cost efficiency by using optimized I/O operations for high performance.
- **Data Transfer:** Data transfer between Amazon Aurora and other AWS services is generally free within the same region and while charges may apply for cross-region data transfer.

Amazon Aurora



Advantages of Amazon Aurora

- **Security:** Aurora is service from Amazon, the user is assured about the security and can use the IAM features.
- **Availability:** Multiple replications of DB instance, over numerous zones guarantees high accessibility.
- **Scalability:** With Aurora serverless, the user can set-up the database to automatically scale up and scale down with application demand.
- **Performance:** With simplicity & cost-adequacy as open-source database.
- **Upkeep:** Aurora has zero server maintenance. 5 times faster than MySQL and 3 times faster than PostgreSQL
- **Management Console:** Amazon Management Console is easy to use and drag features to immediately set-up the Aurora Cluster.

Limitation of Amazon Aurora

- The primary limitations of Amazon Aurora for a typical user are its higher cost compared to standard RDS, vendor lock-in to the AWS ecosystem, and a single point of write operations which can bottleneck extremely heavy, write-intensive application

Amazon DynamoDB



- Amazon DynamoDB is a fully managed, serverless NoSQL database service offered by Amazon Web Services (AWS). It is designed to deliver fast and predictable performance with seamless scalability, making it suitable for a wide range of applications that require low-latency data access at any scale.

Key characteristics of Amazon DynamoDB:

- **Fully Managed and Serverless:** DynamoDB eliminates the need for users to provision or manage servers, operating systems, or database software. AWS handles all the underlying infrastructure, including patching, backups, and scaling.
- **NoSQL Database:** Unlike traditional relational databases, DynamoDB is a NoSQL database, meaning it does not adhere to a rigid, predefined schema. It supports key-value and document data models, offering flexibility in how data is structured.
- **High Performance and Scalability:** DynamoDB is built for high-performance applications, providing consistent single-digit millisecond latency even at massive scales. It automatically scales capacity to accommodate varying workloads.

Amazon DynamoDB



Key characteristics of Amazon DynamoDB:

- **Durability and High Availability:** Data in DynamoDB is automatically replicated across multiple Availability Zones within an AWS Region, ensuring high durability and availability and protecting against data loss.
- **Flexibility:** DynamoDB's schema-less nature allows for easy evolution of data models, making it adaptable to changing application requirements.
- **Integration with AWS Services:** It integrates seamlessly with other AWS services, simplifying the development of complex applications.
- **Features:** DynamoDB offers features like secondary indexes for flexible querying, on-demand backup and restore, and change data capture through DynamoDB Streams.

Amazon DynamoDB

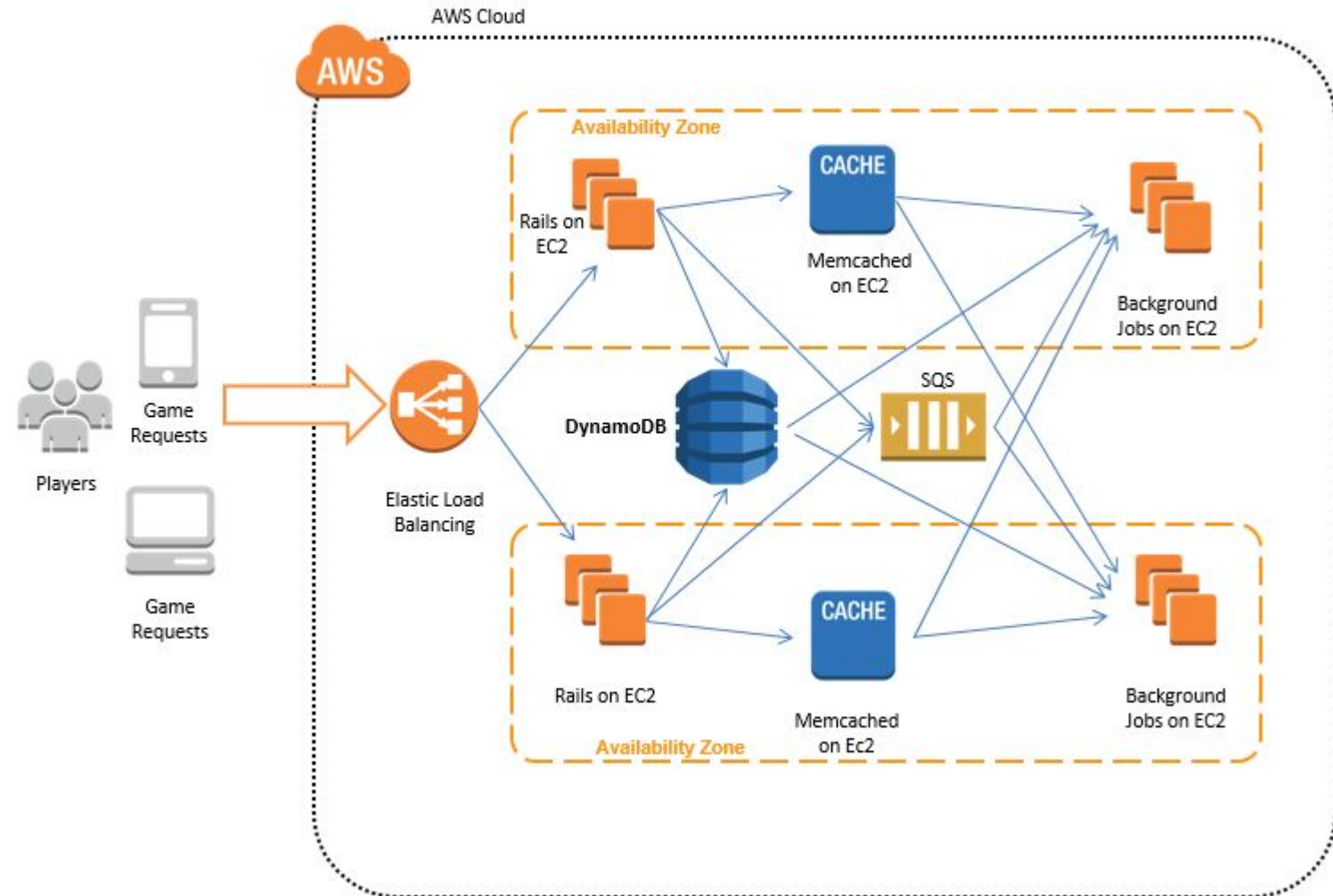


Amazon DynamoDB is a cloud-native NoSQL primarily key-value database.

- namoDB is **cloud-native** in that it does not run on-premises or even in a hybrid cloud; it only runs on Amazon Web Services (AWS). This enables it to scale as needed without requiring a customer's capital investment in hardware. It also has attributes common to other cloud-native applications, such as elastic infrastructure deployment (meaning that AWS will provision more servers in the background as you request additional capacity).
- DynamoDB is **NoSQL** in that it does not support ANSI Structured Query Language (SQL). Instead, it uses a proprietary API based on JavaScript Object Notation (JSON). This API is generally not called directly by user developers, but invoked through AWS Software Developer Kits (SDKs) for DynamoDB written in various programming languages (C++, Go, Java, JavaScript, Microsoft .NET, Node.js, PHP, Python and Ruby).
- DynamoDB is primarily a **key-value** store in the sense that its data model consists of key-value pairs in a schemaless, very large, non-relational table of rows (records). It does not support relational database management systems (RDBMS) methods to join tables through foreign keys. It can also support a document store data model using JavaScript Object Notation (JSON).

Amazon DynamoDB

DynamoDB's NoSQL design is oriented towards simplicity and scalability, which appeal to developers and devops teams respectively. It can be used for a wide variety of semi-structured data-driven applications prevalent in modern and emerging use cases beyond traditional databases, from the Internet of Things (IoT) to social apps or massive multiplayer games. With its broad programming language support, it is easy for developers to get started and to create very sophisticated applications using DynamoDB.



Amazon DynamoDB

Key Features



1. Fully Managed Service:

- [Amazon DynamoDB](#) is a fully managed serverless database.
- DynamoDB handles setup, infrastructure, maintenance, and scaling automatically.

2. NoSQL Data Model:

- In DynamoDB, data can be stored and retrieved as key-value pairs, documents, or wide column stores.
- It is a fast and flexible NoSQL database service.

3. Scalability:

- DynamoDB is a fully managed serverless database that automatically scales to fit developer's needs.
- DynamoDB scales to zero.
- It has no cold starts, no version upgrades, no maintenance, no downtime.
- DynamoDB global tables is a multi-region, multi active database with 99.999% availability.

Amazon DynamoDB

Key Features

4. Consistency Models:

- DynamoDB offers a two-consistency model:
 - Strongly Consistent Reads.
 - Eventually Consistent Reads.
- It consistently handles more than 10 trillion request per day.
- DynamoDB allowing developers to choose between strict consistency or high performance depending on their requirements

5. Flexible Querying:

- DynamoDB supports various querying methods, containing key-value lookups, range queries,
- Secondary indexes which is optional and enabling efficient data retrieval based on different access patterns.

6. Backups:

- Point-in-time [PITR] recovery provides continuous backup and protect DynamoDB tables data from accidental or delete operations
- OnDemand backup and recover provides full backup of DynamoDB tables data.
- Developer can backup data from a few megabytes to hundreds of terabytes of data.
- The Backup and restore operation do not affect DynamoDB's performance.



Amazon DynamoDB



Key Features

7. Integrated with AWS Ecosystem:

- DynamoDB is a part of [Amazon](#) ecosystem.
- The platform allows developers to build serverless applications, leverage infrastructure code, and
- build real-time data-driven applications through integrations with other services such as AWS Lambda, AWS CloudFormation, and AWS AppSync.

8. On-Demand and Provisioned Throughput:

- Amazon DynamoDB has two pricing models, pricing for on-demand capacity mode and pricing for provisioned capacity mode.
- [On-demand capacity](#) mode: Simplified billing, pay only for the read and writes your application performs.
- Provisioned capacity mode: Manage and optimize the pricing by allocating reads and writes capacity in advance.

9. Highly Available:

- In a Region, DynamoDB replicates data across multiple availability zones, ensuring high availability.
- DynamoDB automatically handles replication, recovery, reducing the possibility of data loss or service disruption.
- The availability SLA of DynamoDB is up to 99.999%.

Amazon DynamoDB Streams



- DynamoDB Streams is a feature of Amazon's DynamoDB database that essentially acts like a real-time logbook of all changes happening within your DynamoDB table.
- Imagine you have a notebook where you track all your inventory. Every time you add a new item, sell an item, or update an item's details, you write it down in chronological order. DynamoDB Streams works similarly for your database.
- **How it works:** These changes are recorded in a time-ordered sequence, like pages in a logbook. Each entry in this logbook contains details about the specific change, such as what was added, what was changed (and what it looked like before and after), or what was deleted.

Why Amazon DynamoDB Streams?



- Other applications or services can "read" this logbook in near real-time. This allows them to react to changes in your DynamoDB table automatically. For example:
- If a new user signs up (an item is added to a "users" table), you could trigger an email to welcome them.
- If a product's price changes, you could automatically update a related inventory system or send a notification.
- You could use it to replicate data to another database for backup or analytics.

Key Features of Amazon DynamoDB Streams



- Time-ordered: Changes are recorded in the exact order they occurred.
- Near real-time: Applications can access the changes almost immediately.
- Durable: The stream records are stored for up to 24 hours, giving consuming applications time to process them.

In essence, DynamoDB Streams provides a powerful way to integrate your DynamoDB data with other parts of your application or system, enabling automated responses to data changes.

AWS Management Tools



- AWS management tools are a collection of services that help you control, monitor, and automate your cloud resources on Amazon Web Services (AWS). Think of them as a dashboard and a set of tools for a large virtual "computer" or "data center" in the cloud, allowing you to manage things like server setup, security, and costs from a single place.
- **Centralized control:** The main tool is the AWS Management Console, a web-based portal that gives you a single place to see and manage all your services, from storage to databases.
- **Automation:** Tools like AWS CloudFormation let you use simple text files to automatically set up and manage your cloud infrastructure, so you don't have to manually configure every server and service.
- **Monitoring and optimization:** Services like AWS Systems Manager help you keep an eye on your resources, automate tasks like applying security patches, and ensure everything is running efficiently.

AWS Management Tools



- **Cost management:** You can use these tools to track your spending, see where you can cut costs, and avoid overspending on your cloud services.
- **Security and compliance:** They help you manage security credentials, configure permissions, and ensure your setup meets security policies.

AWS Management Tools



- AWS Management Tools are a suite of services designed to monitor, manage, and optimize your cloud resources at any scale, ensuring cost efficiency, performance, and security.
- They help you organize and track your usage of AWS and on-premises infrastructure, enabling you to provision, configure, and automate tasks across your environment.
- Key examples include the AWS Management Console, a web-based platform for managing services; AWS Systems Manager, for operational tasks like patching and configuration management; and AWS CloudFormation, which automates infrastructure deployment through templates.

Key functions and examples



- **Monitoring and logging:**

Amazon CloudWatch: Collects and tracks metrics, collects and monitors log files, and sets alarms for AWS resources and applications.

AWS CloudTrail: Tracks user activity and API calls in your AWS account, providing a history of actions for security and auditing.

- **Automation and configuration:**

AWS Systems Manager: Automates operational tasks such as software inventory, OS patching, and system image creation across your EC2 and on-premises servers.

AWS CloudFormation: Allows you to model and provision your AWS and third-party resources with a text-based template.

AWS OpsWorks: Provides application and server management using automation platforms like Chef and Puppet.

Key functions and examples



- **Optimization and cost management:**

AWS Trusted Advisor: Provides recommendations for cost savings, performance, security, and fault tolerance.

AWS Cost Explorer: Helps you analyze your cost and usage to identify trends and potential savings.

- **Centralized access:**

AWS Management Console: A web-based application providing a graphical user interface to access and manage all AWS services from a single platform.

AWS Systems Manager Explorer: A customizable dashboard that provides a view of the operational health of your AWS environments.

AWS CloudWatch



- AWS CloudWatch is like a centralized dashboard and alarm system for all your cloud services.
- It automatically collects performance data (like CPU usage or network traffic) and logs from your AWS resources, allowing you to see how they are doing all in one place.
- You can then use this information to create visual charts to understand performance, set up alerts to notify you of problems, and even trigger automated actions, like adding more servers when traffic is high.

Key Functions of AWS CloudWatch



- Collects and stores data: CloudWatch acts as a single repository for performance metrics and logs from a wide range of AWS services and your own applications.
- Monitors performance: It helps you track key performance indicators, such as CPU utilization, network traffic, and disk operations, to ensure your applications are running smoothly.
- Creates alarms: You can set up alarms to notify you when a specific metric crosses a threshold (e.g., when CPU usage goes above 80%).
- Automates actions: These alarms can be configured to trigger automated actions, such as starting or stopping an EC2 instance, or scaling your resources up or down using services like EC2 Auto Scaling.
- Provides visibility: It gives you a unified view of your entire infrastructure, so you can see how different parts are performing together instead of looking at them in silos.

Advantages of AWS CloudWatch



- **Centralized and real-time monitoring**

Unified view: CloudWatch provides a single platform to monitor all your AWS resources and applications, including on-premises environments.

Real-time data: It collects and displays metrics and logs in near real-time, allowing for quick identification of performance bottlenecks or issues.

Detailed insights: You can view detailed metrics like CPU utilization, memory usage, and request latency, along with logs from various sources.

- **Proactive issue detection and automated response**

Alarms and notifications: Set up alarms to monitor CloudWatch metrics and trigger automated actions or send notifications when thresholds are breached.

Automated actions: These alarms can be configured to automatically perform tasks like rebooting instances or triggering auto-scaling events, ensuring high availability.

Advanced troubleshooting: Features like ServiceLens and Synthetics help monitor application health and endpoints, while Log analytics provides advanced querying of log data for troubleshooting without needing extra servers.

Advantages of AWS CloudWatch



- **Cost optimization and performance improvement**

Cost management: By monitoring resource utilization, you can identify underused or overprovisioned resources and optimize your cloud spending.

Performance enhancement: It provides the data needed to analyze performance trends and make informed decisions to optimize both application and resource performance.

Resource planning: Features like auto-scaling and historical data analysis help in planning for capacity and managing resources efficiently.

- **Scalability and integration**

Scalable service: CloudWatch is designed to be a scalable service, capable of monitoring large and complex environments.

Wide integration: It integrates with over 70 AWS services, allowing you to collect a comprehensive view of your entire AWS environment on one platform.

Third-party integrations: It can be integrated with third-party tools like Grafana and PagerDuty to enhance your monitoring capabilities.

Disadvantages of AWS CloudWatch



- Disadvantages of AWS CloudWatch include its complexity and high cost, especially for large-scale or high-resolution monitoring.
- It can also be difficult to use for cross-region or multi-account monitoring and has limitations in real-time data visibility and flexible visualization.
- For microservices architectures, tracking logs across different services can be tedious, and data-driven decision-making can be limited by the inflexibility of the tool.

Complexity and steep learning curve

- CloudWatch can be complex to set up and configure, particularly for large or intricate infrastructures.
- Mastering the management of metrics, alarms, and dashboards requires significant time and effort.

Cost

- Pricing can become expensive, especially for large-scale applications, due to costs associated with data ingestion, storage, and access.
- Costs can increase quickly with the use of high-resolution custom metrics or extensive data collection.

Limitations of AWS CloudWatch



- Cross-region/multi-account monitoring: CloudWatch is region-specific by default and requires manual configuration with tools like EventBridge or Lambda for cross-region views.
- Real-time visibility: Standard metrics have a 1-minute granularity, and logs can take seconds to minutes to appear, which may be too slow for real-time troubleshooting.
- Customization and visualization: Some users find limitations in customizing dashboards and have a preference for more flexible and visually appealing options, as found in other tools.
- Microservices: It can be tedious to track logs across multiple microservices, which can limit visibility and troubleshooting.
- Non-AWS resources: While it can monitor non-AWS resources, its primary design is for AWS resources, and it may be less effective in hybrid environments.

AWS CloudWatch Metrics



- CloudWatch metrics are time-ordered data points that represent variables to monitor, such as CPU usage or request counts, to provide insights into the performance of AWS resources and applications. These metrics are used to create dashboards, set alarms for automated actions, and perform detailed analysis to troubleshoot issues and optimize performance.
- CloudWatch Metrics are like a health report for your Amazon Web Services (AWS) applications and resources, using numbers and graphs to show how they're performing in real-time. They are a series of data points, like CPU usage or the number of requests, that are collected over time to show trends and help you spot issues or areas for improvement.

AWS CloudWatch Metrics



Key aspects of CloudWatch metrics

- **Data points:** A metric is a variable with a series of data points, each with a timestamp and a value. For example, a data point for "CPU Utilization" would be the percentage of CPU used at a specific time.
- **Standard metrics:** These are pre-configured metrics provided automatically by AWS services like EC2, RDS, and ELB, covering aspects like CPU utilization, network traffic, and disk I/O.
- **Custom metrics:** You can also publish your own data points to CloudWatch to monitor custom application-level performance data, such as memory utilization on an EC2 instance, using the CloudWatch agent.
- **Dimensions:** These are name/value pairs that are part of a metric's identity, allowing you to add a unique characteristic. For example, you could add a "InstanceId" dimension to a CPU utilization metric to specify a particular EC2 instance.

AWS CloudWatch Metrics



Key aspects of CloudWatch metrics

Monitoring and analysis:

- **Dashboards:** You can create dashboards to visualize metrics and see a unified view of your resources.
- **Alarms:** You can set up alarms to automatically trigger actions, such as sending a notification or scaling resources, when a metric breaches a defined threshold.
- **Metrics Insights:** This is a high-performance query engine that uses a SQL-based language to analyze and aggregate metrics at scale, which helps in faster troubleshooting.
- **Granularity and resolution:** CloudWatch offers different monitoring options. Basic monitoring (free) publishes metrics at 5-minute intervals, while detailed monitoring (paid) publishes them every minute. High-resolution metrics are also available, with a granularity of 1 second.

Common AWS Service Namespaces and Associated Metrics



- AWS/EC2: Metrics related to EC2 instances (CPU Utilization, Network In/Out, Disk Read/Write Ops).
- AWS/EBS: Metrics for Elastic Block Store volumes (Volume Read/Write Bytes, Volume Idle Time).
- AWS/RDS: Metrics for Relational Database Service instances (CPU Utilization, Database Connections, Free Storage Space).
- AWS/Lambda: Metrics for Lambda functions (Invocations, Errors, Duration, Throttles).
- AWS/S3: Metrics for S3 buckets (BucketSizeBytes, NumberOfObjects, AllRequests, GetRequests).
- etc.,

CloudWatch Alarms



- AWS CloudWatch alarms are like a digital security guard for your applications that watches over key performance indicators (KPIs) and sends alerts when things get bad.
- You set a specific threshold for a metric, like the "CPU usage" of a server, and if the usage goes above that threshold for a set period, the alarm triggers an action.
- This action can be as simple as sending an email or SMS to your team, or more complex, like automatically running a script to restart a server or provision more storage.

How CloudWatch Alarms Work?



- Watch a metric: You choose a specific metric to monitor, like CPU utilization, how full a storage drive is, or the number of customer requests your app is handling.
- Set a threshold: You define a "normal" level for that metric. For example, you might set an alarm if CPU usage goes above 80%.
- Choose a period: You specify how many data points in a row need to be over the threshold before the alarm is triggered. For instance, you can set it to trigger only if the CPU usage is over 80% for three consecutive minutes.
- Define an action: You decide what happens when the alarm is triggered.

Actions can include:

- Sending a notification via email or text message using Amazon Simple Notification Service (SNS).
- Running a function to perform a task, such as automatically adding more storage to a database.
- Triggering a more complex workflow or creating a ticket in a service management tool.

Examples of what you can do



- **Prevent performance issues:** Get an alert when your server's CPU is getting too high, so you can investigate before it slows down or crashes.
- **Automate tasks:** Automatically add more storage to a database when the storage space drops below a certain level.
- **Improve security:** Get an alert if a security group is changed, and use an automation to isolate the affected resources.
- **Combine alarms:** Create a "composite alarm" that triggers only if multiple conditions are met, such as high CPU and low memory, to reduce false alarms.

AWS CloudTrail



- AWS CloudTrail is like a security camera and a detailed diary for your entire AWS account. It continuously monitors and records almost every action taken by users, roles, or AWS services, providing a clear record of "who did what, when, and from where".
- AWS CloudTrail is a service that enables auditing, compliance, and governance for your AWS account by logging every API call made within it.
- It records who made the request, what service was used, the action taken, and when it happened, providing a detailed history of activity across your AWS environment.
- This helps with security analysis, resource change tracking, and meeting compliance requirements.

What does AWS CloudTrail do?



- Records API activity: CloudTrail logs actions from the AWS Management Console, Command Line Interface (CLI), SDKs, and other AWS services.
- Provides details for auditing: Each event log includes information like the user or role that made the request, the source IP address, the time of the call, and the parameters used.
- Supports governance and compliance: By providing a complete history of activity, CloudTrail helps organizations ensure they meet compliance standards and maintain governance over their cloud resources.
- Enables security analysis and troubleshooting: The logs are essential for tracking changes, investigating security incidents, and troubleshooting issues by showing who did what and when.
- Offers different ways to use the data: You can view a searchable history of the last 90 days, use CloudTrail Lake to store and analyze events in a data lake, and integrate it with other services.

Limitations of AWS CloudTrail



- AWS CloudTrail has limitations including unconfigurable API call limits, a maximum of 5 trails and 5 event data stores per region, and a limit of 5 event selectors per trail.
- It also has a maximum event size of 256 KB when logging to CloudWatch Logs or EventBridge.
- Other limitations include limits on the total number of data resources across an account, the requirement for opt-in for data and network events, and a cost associated with data and insights events.

Limitations of AWS CloudTrail



- **Resource and configuration limits**
- **Trails:** A maximum of 5 trails can be created per AWS region. A trail that applies to all regions counts as one trail in each region.
- **Event Data Stores:** You can create up to 5 event data stores per region in CloudTrail Lake.
- **Event Selectors:** An individual trail can have up to 5 event selectors. An advanced event selector has a limit of 500 conditions across all advanced event selectors in a trail.
- **Data Resources:** The total number of data resources across all event selectors in a trail is limited to 250. However, an individual event selector can have up to 250 data resources if the total limit is not exceeded.
- **Event Size:** Events over 256 KB cannot be sent to CloudWatch Logs or EventBridge.

Limitations of AWS CloudTrail



- **API and performance limits**
- API Rate Limits: API limits are set and cannot be changed.
- 10 TPS: for Get, describe, and list APIs.
- 2 TPS: for the LookupEvents API.
- 1 TPS: for all other APIs.
- **Logging and cost limitations**
- Data and Network Events: Data and network events are not enabled by default due to their high volume and cost. They must be explicitly enabled through trails or event data stores.
- Insights Events: CloudTrail Insights, which detect unusual activity, are not enabled by default and incur an additional charge.
- Event History: The free, 90-day event history view in the console is limited to management events and does not include data events.

AWS Config



- AWS Config is a service that enables you to create an inventory of your AWS resources, track their configurations, and view the history of changes over time. It helps you perform security analysis, compliance auditing, and troubleshooting by providing a detailed view of how your resources are configured and related to one another. This allows you to see how configurations and relationships have changed over time.
- **Key features**
- Resource inventory: It provides a complete inventory of your resources, including third-party resources.
- Configuration history: It records the history of your resource configurations, so you can see how they were configured at any point in the past.
- Change notifications: It can send notifications when a resource's configuration changes.
- Compliance auditing: It helps you evaluate the configurations of your AWS resources to ensure compliance with security and operational best practices.

AWS Config Example



- AWS Config is like a security camera and rule enforcer for your AWS cloud environment.
- Imagine you have a house with many valuable items. AWS Config does two main things for your "cloud house":
- It constantly records everything that happens to your items (AWS resources). Did you add a new server? Change a setting on a database? Make a storage bucket public? AWS Config keeps a detailed history of every change, who made it, and when. This is like a security camera recording every movement in your house.
- It checks if your items are following your rules. You can set up rules like "all doors must be locked" (e.g., all S3 buckets must be private) or "only authorized people can enter certain rooms" (e.g., only specific users can access certain databases). AWS Config automatically checks if your resources are compliant with these rules and alerts you if they're not. This is like having a guard who ensures everyone follows the rules you've set for your house.

Difference between AWS CloudTrail, AWS CloudWatch and AWS Config



AWS CloudTrail, CloudWatch, and Config are all AWS services designed to help manage and monitor your AWS environment, but they serve distinct purposes:

- CloudTrail records every single API call and action made within your AWS account. This includes who did what, when, from where, and on which resource.
- CloudWatch collects metrics (data points about performance, like CPU usage, network traffic, or disk I/O) and logs from your AWS resources and applications. You can use this data to create dashboards, set alarms, and trigger automated actions.
- AWS Config keeps a detailed inventory of all your AWS resources and tracks how their configurations change over time. It can also evaluate these configurations against predefined rules to ensure compliance and identify misconfigurations.

AWS Systems Manager



- AWS Systems Manager is a management service that helps you control and automate your AWS and hybrid cloud resources. AWS Systems Manager is a management service for automating tasks like patching, configuration, and software deployment across AWS and hybrid cloud environments. It's a broad tool that differs from services like CloudWatch, which focuses on monitoring, or Secrets Manager, which is specifically for storing secrets. Key features of Systems Manager include secure, remote access to instances without a bastion host, and the ability to manage diverse resources like EC2 instances and on-premises servers.
- **Primary Function:** Manages, automates, and enforces configurations across your resources.
- **Key Tasks:** Patching, installing software, running commands remotely, and creating system images.
- **What it's used for:** To simplify managing servers and applications at scale, both in the cloud and on-premises.
- **Example:** Using Systems Manager's Patch Manager to automatically apply security patches to all your servers.

Key functions of AWS Systems Manager



- **Automation:** Automates routine operational tasks such as patching, software installation, and configuration changes across large groups of instances.
- **Unified view:** Provides a centralized console to view and manage resources, including the ability to group them by tags for applications, layers, or environments.
- **Secure remote access:** Enables secure, remote management of instances without requiring SSH or bastion hosts, and allows for fine-grained control over who can access what.
- **Configuration management:** Helps maintain a desired state by applying configurations and reporting on compliance.
- **Hybrid cloud support:** Manages resources both in AWS and on-premises or in other clouds by registering nodes with the SSM Agent.

Comparison with other services



- **vs. AWS CloudWatch:** CloudWatch is a monitoring service that collects logs, metrics, and events. Systems Manager is a management tool that acts on those resources. You might use CloudWatch to monitor performance and use Systems Manager to automatically patch instances that are showing high CPU usage.
- **vs. AWS Secrets Manager:** Secrets Manager is a service for managing sensitive secrets like database credentials and API keys. Systems Manager includes Parameter Store, which can store similar values, but Secrets Manager provides more advanced features like automatic secret rotation.
- **vs. Microsoft Azure Automation:** Both services automate tasks on their respective cloud platforms. The main difference is the platform; Systems Manager is for AWS and hybrid environments, while Azure Automation is for the Azure platform.



Management & Governance

AWS ELB and Auto Scaling



- **AWS ELB**
- AWS ELB (Elastic Load Balancing) acts as a smart "traffic controller" for your applications running on Amazon Web Services. It automatically distributes incoming internet traffic across multiple servers (like EC2 instances, containers, or IP addresses) to ensure no single server gets overloaded.
- ELB, or Elastic Load Balancing, is a fully managed service from AWS that automatically distributes incoming application traffic across multiple targets, such as EC2 instances, containers, and IP addresses, in one or more Availability Zones. It improves application performance and availability by monitoring the health of registered targets and routing traffic only to healthy ones. The service automatically scales its capacity to handle incoming traffic fluctuations.

AWS ELB and Auto Scaling



How It Works

- Imagine you have a popular restaurant with a team of chefs (your servers) in the kitchen, all capable of making the same food. Without a system, customers might all go to the first chef they see, overwhelming that one person while others are idle.
- An AWS ELB is like a very efficient maître d':
- Single Entrance: All customer requests (traffic) first come to the ELB, which acts as a single point of contact.
- Smart Distribution: The ELB directs each new customer to an available chef in the kitchen using various rules to ensure the workload is balanced.
- Health Checks: The maître d' constantly checks if each chef is ready to take a new order. If a chef is sick or busy, the ELB stops sending customers their way until they are healthy and ready again, ensuring no one gets stuck waiting for service.
- Scalability: If a sudden rush of customers arrives (a traffic spike), the ELB can automatically call in more chefs (scale up your resources) to handle the demand without any manual intervention, then send them home when it's quiet again.

AWS ELB and Auto Scaling



Key functions of ELB

- **Traffic distribution:** ELB acts as a single point of contact for clients and intelligently routes requests to healthy backend targets to prevent any single instance from being overwhelmed.
- **Automatic scaling:** It automatically adjusts its own capacity to handle changes in traffic, ensuring a smooth user experience even during high-demand periods.
- **Health checks:** ELB continuously monitors the health of registered targets and automatically stops sending traffic to any that become unhealthy.
- **High availability:** By distributing traffic across multiple targets and Availability Zones, ELB helps create a highly available and fault-tolerant application architecture.
- **Integration with other services:** It can be integrated with services like Amazon EC2 Auto Scaling to automatically register and deregister instances as they are launched or terminated, simplifying management.

AWS ELB and Auto Scaling



AWS Auto Scaling

- AWS Auto Scaling automatically adjusts the number of server resources to match your application's demand, ensuring performance during high traffic and saving costs during low traffic. It works by monitoring your resources, adding more servers when needed (scaling out), and removing them when they aren't (scaling in). This prevents your application from running slowly due to a lack of power and ensures you aren't paying for idle resources.
- AWS Auto Scaling automatically adjusts the number of EC2 instances and other resources to handle your application's fluctuating demand, which improves performance and availability while managing costs. It works by monitoring your resources and adding capacity during peak times and removing it when demand decreases. This can be configured through scaling plans, which allow for simplified setup using predefined strategies or custom policies to optimize for availability, cost, or a balance of both.

AWS ELB and Auto Scaling



How it works

- **Monitors resources:** AWS Auto Scaling continuously monitors the performance of your applications.
- **Adjusts capacity:** When it detects that demand is increasing or decreasing, it automatically adds or removes resources to meet the required capacity.
- **Defines scaling policies:** You can create scaling policies that specify how the system should react to changes in demand. These policies can be set to be predictive or reactive.
- **Manages multiple services:** It can be used to manage scaling for a variety of AWS services, not just EC2, including ECS, DynamoDB, and EMR.

AWS ELB and Auto Scaling



Benefits of Auto Scaling

- **High availability:** Ensures that you have enough capacity to handle traffic spikes, preventing downtime.
- **Cost optimization:** Reduces costs by automatically scaling down resources when they are not needed.
- **Flexibility and efficiency:** Allows for better flexibility and reliability by adapting resources in real-time to the application's needs.
- **Simplified management:** Scaling plans can be used to configure and manage scaling for multiple resources across different services from a single interface.

AWS ELB and Auto Scaling



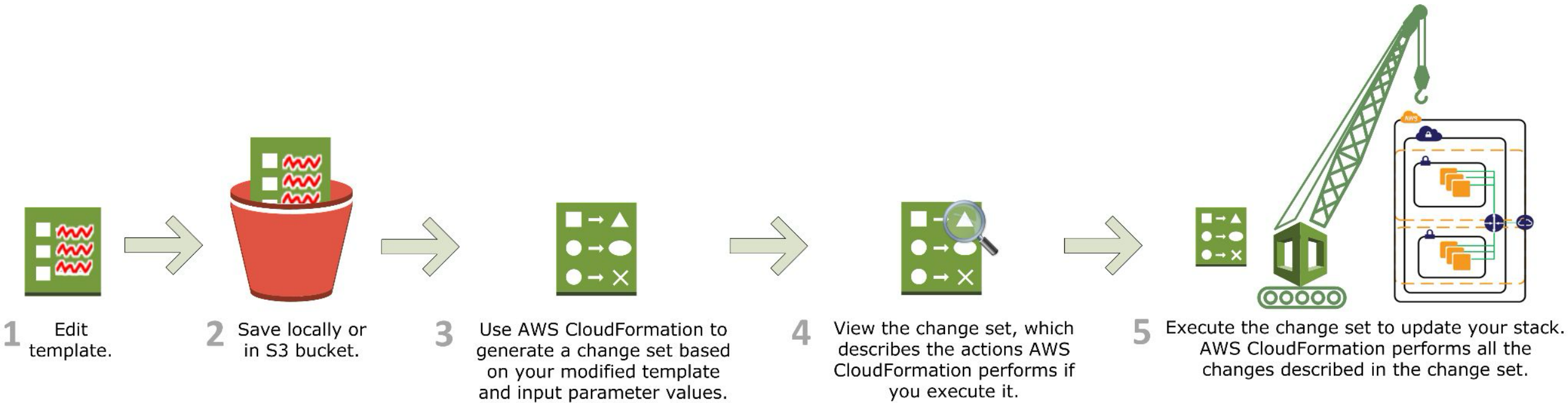
Difference between AWS ELB and AWS Auto Scaling

- AWS ELB distributes incoming traffic to healthy instances, while AWS Auto Scaling automatically adds or removes EC2 instances to match demand.
- ELB ensures a single instance isn't overwhelmed by balancing load, and it can reroute traffic away from unhealthy instances. Auto Scaling adjusts the number of instances to maintain optimal performance, responding to metrics like CPU utilization.
- They often work together, with ELB distributing traffic to the instances that Auto Scaling manages.

AWS CloudFormation

- CloudFormation is like a blueprint for your cloud-based computer setup that you write in a text file, which you then give to Amazon Web Services (AWS) to automatically build.
- Instead of manually creating each server, database, and storage bucket, you write a template that describes all the parts you need, and CloudFormation handles the setup in the correct order, making it fast, reliable, and repeatable.
- This is called "Infrastructure as Code" because you are treating your infrastructure the same way you would write code.
- CloudFormation is an AWS service that uses Infrastructure as Code (IaC) to automate the provisioning and management of AWS resources.
- It allows you to model and set up your infrastructure by writing a template in JSON or YAML that describes the desired resources, such as EC2 instances or S3 buckets, and CloudFormation handles creating, configuring, and updating them for you.

AWS CloudFormation



AWS CloudFormation



How it works

- **Create a template:** You write a text file in either JSON or YAML that describes all the AWS resources you need. This template acts as a single unit, referred to as a "stack".
- **Provision resources:** You provide the template to CloudFormation, which then automatically handles the provisioning and configuration of the resources you specified, including managing dependencies between them.
- **Manage infrastructure:** You can update, replicate, or delete resources as a single unit (the stack) by simply modifying and reapplying your template.

Key Benefits of CloudFormation

- **Automation:** Automates the creation and management of infrastructure, reducing the time and effort required to manage resources.
- **Consistency:** Ensures that your infrastructure is provisioned in a consistent and repeatable way across different environments, such as development, testing, and production.
- **Reliability:** By treating infrastructure as code, you can reduce manual configuration errors and manage your infrastructure through a reliable and predictable process.
- **Simplified updates:** Allows you to make incremental changes to your infrastructure by creating "change sets" that show a preview of what will be updated before you apply the changes.



CloudTrail and CloudWatch in Slides 38-55.



Thank You!